Parallel Prefix Sums: Work Efficiency and Thread Scaling

CIS 431/531 — Homework 02

1 Algorithms

For input x, all implementations compute the inclusive sum $y_i = \sum_{j=0}^{i} x_j$ using the starter's 32-bit `int` data. The serial recurrence takes $n-1$ additions. The first parallel implementation doubles its offset $d = 1, 2, 4, \ldots$: each level reads the previous buffer and writes $y_i = x_i + x_{i-d}$ where the predecessor exists. Separate buffers and a barrier between levels avoid read/write races. Its work is $O(n \log n)$ and span $O(\log n)$.

The second implementation uses the lecture's inclusive binary-tree up-sweep/down-sweep. Upward stages form subtree sums; downward stages propagate accumulated prefixes to the remaining positions. For $n = 2^k$, it performs $(n-1)+(n-1-k)$ additions: $O(n)$ work with $O(\log n)$ span. Non-power-of-two lengths skip nonexistent positions. Both parallel implementations keep one OpenMP team across all stages.

2 Experimental method

Platform: Talapas, Slurm job 48205110 on n0187; AMD EPYC 7713 64-Core Processor; GCC 8.5.0 with `-O3 -std=c99 -fopenmp` (C++17 measurement driver). **Execution status:** Measured on the compute node inside the recorded Slurm allocation. Allocation: 64 CPUs and 4 GiB RAM.

We tested $n = 2^{16}, 2^{20}, 2^{24}, 2^{25}$ with 1, 2, 4, 8, 16, 32, 64 threads. Inputs $x_i = (i \bmod 201) - 100$ keep intermediate sums in range. Each configuration received one excluded warmup and 7 measured trials, shuffled in a reproducible order: 420 measurements across 60 configurations. Monotonic wall-clock timings are used. The figure shows median speedup and the interquartile range of runtimes transformed to speedup using the serial median. Memory allocation, input generation, validation, and CSV output are excluded; algorithm-internal copies, synchronization, team overhead, and any final buffer copy are included. All arrays use the same parallel first-touch policy. Threads are bound with `OMP_PLACES=cores` and `OMP_PROC_BIND=close`. Every measured output is checked against the serial result outside the timed region. Independent `std::partial_sum` correctness tests cover signed/zero/large values, empty input, and irregular lengths. As in the starter, preparing a disposable input copy is outside timing; the doubling method reuses that copy as scratch storage. Its initial output copy and final buffer copy remain inside timing.

3 Results and interpretation

For $n = 33,554,432$, the serial median was 15.61 ms. The best tree result was 18.34 ms at 64 threads (0.85× speedup); the best doubling result was 98.12 ms at 64 threads (0.16×). Table 1 summarizes the best observed speedup for each input size. These are observed optima among the tested thread counts. At 64 threads, tree and doubling speedups were 0.85 and 0.16, respectively.

Neither parallel method's median time beat serial at any tested input size. Lower total work alone did not guarantee proportional thread scaling. Tree traversal has strided access and barriers, while doubling performs many full-array passes. Untimed source-copy preparation can also favor serial cache locality. These are plausible contributors to the observed timings; hardware-counter measurements did not isolate their effects. The node has 128 physical cores across two sockets and eight NUMA domains. The shared first-touch policy and core binding can affect serial/parallel ratios; this experiment does not isolate placement effects or characterize other nodes.

Table 1: Best speedup over serial; thread count in parentheses.

n	$O(n \log n)$ doubling	$O(n)$ tree
65,536	0.16 (8)	0.56 (8)
1,048,576	0.18 (16)	0.79 (8)
16,777,216	0.19 (32)	0.80 (32)
33,554,432	0.16 (64)	0.85 (64)

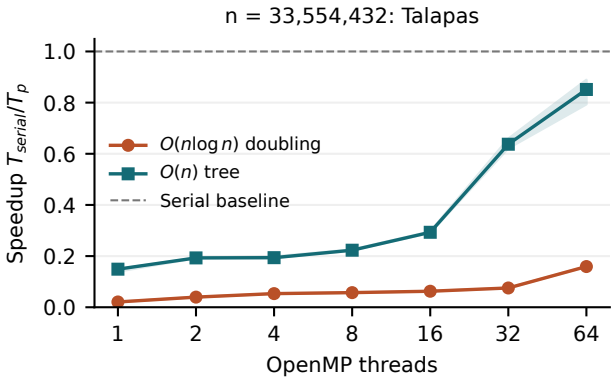


Figure 1: Thread scaling at the largest input. Shading reflects runtime IQR; the dashed line is serial performance.

Provenance

Algorithms follow lecture 7, slides 43–64, and the current course starter. The provided serial recurrence is retained, with an empty-input guard. Code, experiment scripts, and this report were prepared with AI assistance. The supplied course slides exclude AI coding help for homework; verify permission before submission. Raw timings and environment records are included with the code.